

AiroEmed Backend Architecture

Field	Details
Document	Backend Architecture
Objective	Describe how the AiroEmed backend is structured — portals, auth, payments, storage, caching, and fulfillment assignment — for the client tech/product team.
Audience	Client tech/product team; delivery and integration stakeholders
Prepared by	AndOr Communications Pvt Ltd
Classification	Confidential — for client project stakeholders
Scope	telemed-backend API architecture and integrations
Out of scope	Patient journey narrative (see BUSINESS_LOGIC); detailed request/response contracts (see INTEGRATION / API)
Version	1.0
Last updated	27 August 2026
Related documents	BUSINESS_LOGIC.md (BUSINESS_LOGIC.md) · INTEGRATION.md (INTEGRATION.md) · API.md (API.md) · DATABASE_TABLES.md (DATABASE_TABLES.md)
PDF	ARCHITECTURE.pdf

Documentation map

Doc	Purpose
BUSINESS_LOGIC.md (BUSINESS_LOGIC.md)	What the platform does — flows, rules, doctor/pharmacy assignment, admin roles
ARCHITECTURE.md (this doc)	How the backend is built — stack, portals, auth, integrations
INTEGRATION.md (INTEGRATION.md)	Patient app API reference
API.md (API.md)	Full route catalog (all portals)
DATABASE_TABLES.md (DATABASE_TABLES.md)	Table reference (internal)

Related setup: [../api/README.md](#) — local env, Postman (ops).

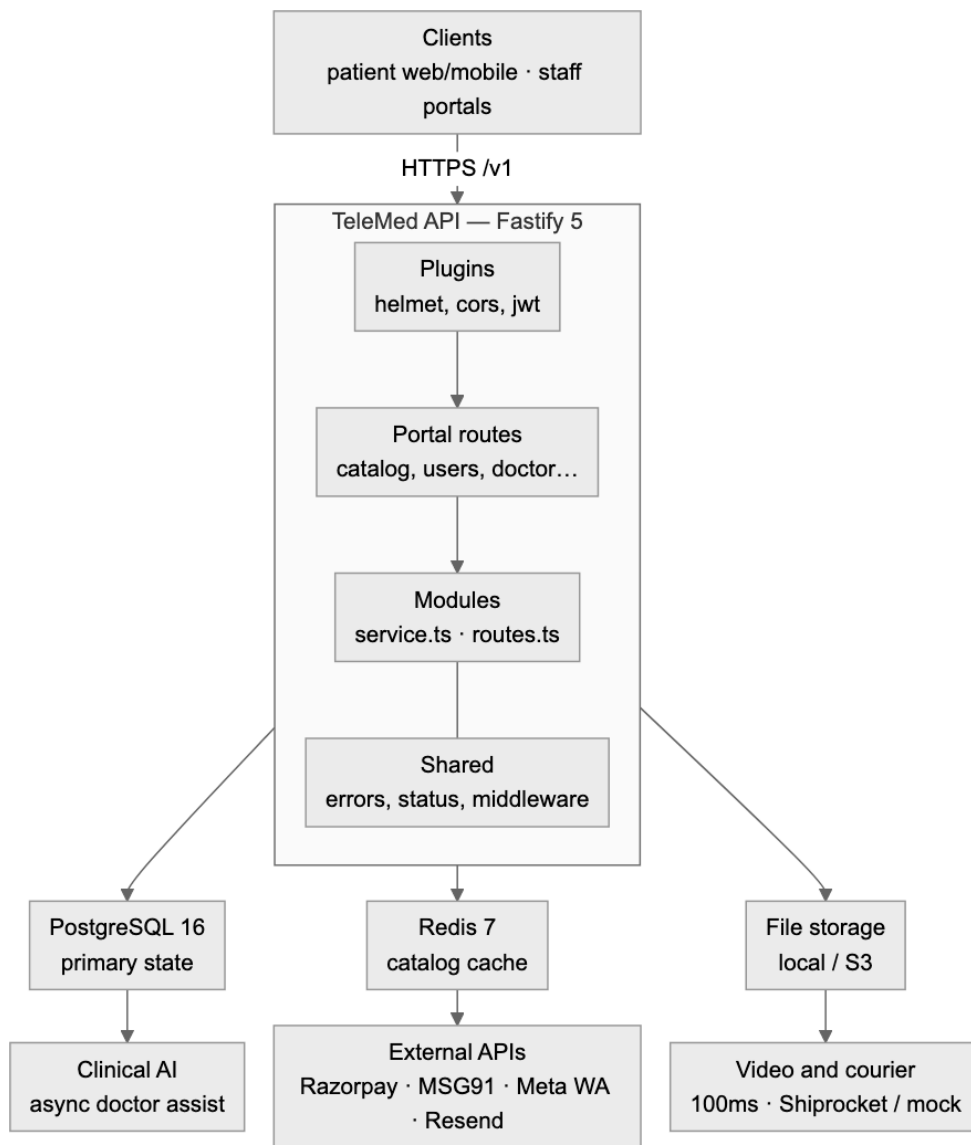
1. High-level system

Production URLs

Service	URL
API	https://api.airoemed.com
Patient website	https://airoemed.com
Admin portal	https://admin.airoemed.com
Catalog CDN	https://cdn.airoemed.com

Applications

Codebase	Apps	API usage
telemed-airohealth	Patient web + mobile	/v1/catalog , /v1/users , auth, notifications
telemed-frontend	Admin, doctor, pharmacy staff (web + mobile)	/v1/admin , /v1/doctor , /v1/pharmacy



Clinical AI (`apps/clinical-ai` , separate service): async doctor-assist for hair, weight, and sexual wellness categories. Queued after RX consultation creation; results are doctor-facing.

Design principles

- **Monolithic REST API** — one deployable `telemed-api` service; domain logic in modules, not microservices.
- **PostgreSQL is source of truth** — Redis caches public catalog/CMS reads ($\geq 1h$ TTL); clear via admin API / version bump (see §14).
- **Portal-oriented routing** — URL prefix maps to audience (`/v1/catalog` , `/v1/users` , ...), not to DB tables. All versioned routes under `/v1/` (no v2).
- **Thin routes, fat services** — HTTP validation in `routes.ts` + Zod; business rules in `service.ts` .
- **Backend-owned presentation fields** — order `displayStatus` and `timeline` are computed server-side so clients do not infer status from raw DB enums. See `BUSINESS_LOGIC.md` §8 (`BUSINESS_LOGIC.md`).

2. Runtime stack

Layer	Technology
HTTP server	Fastify 5
Language	TypeScript (ES modules, <code>tsx</code> in dev)
Validation	Zod on request bodies/queries
Database	PostgreSQL 16 via <code>pg</code> connection pool
Cache	Redis 7 via <code>ioredis</code> — public catalog/CMS (<code>api/src/cache/</code>)
Auth	<code>@fastify/jwt</code> — HS256 access JWT + opaque refresh tokens
Passwords	<code>bcryptjs</code> (cost 12)
Security headers	<code>@fastify/helmet</code>
CORS	<code>@fastify/cors</code> — origins from <code>CORS_ORIGINS</code>
Media	Local filesystem or AWS S3 (<code>@aws-sdk/client-s3</code>)
PDFs	<code>pdfkit</code> — receipts/invoices
Payments	Razorpay (checkout + webhooks) + dummy gateway for dev
OTP / SMS	<code>OTP_PROVIDER</code> : <code>mock</code> (DB) or <code>msg91</code>
Notifications	Multi-channel (<code>whatsapp</code> , <code>email</code> , <code>sms</code> , <code>push</code>) via pluggable providers
Config	<code>dotenv</code> + Zod-validated <code>config/env.ts</code>

Entry point: `api/src/index.ts` → `buildApp()` → listen on `PORT` (default 3000).

3. Repository layout

```

telemed-backend/
├── api/                                # Node API application
│   └── src/
│       ├── app.ts                     # Fastify bootstrap + global plugins
│       ├── index.ts                   # Process entry, graceful shutdown
│       ├── config/env.ts              # Typed environment
│       ├── db/pool.ts                 # PostgreSQL pool
│       ├── plugins/                   # auth, error-handler
│       └── routes/                     # Portal composition (catalog, users,
admin, ...)
│   ├── modules/                       # Domain modules (see §4)
│   │   ├── auth/                      # login, OTP, email verification
│   │   ├── notifications/             # outbound comms + delivery log
│   │   └── purchases/                 # payments, Razorpay webhooks
│   └── shared/                         # Cross-cutting helpers
├── database/
│   └── initial.sql                     # Canonical schema (single source of
truth)
├── seed-dummy-data.sql                 # Local dev seed
└── docs/                               # Technical documentation

```

Module pattern

Each domain area under `api/src/modules/<name>/` typically contains:

File	Responsibility
<code>routes.ts</code> / <code>*-routes.ts</code>	HTTP handlers, <code>preHandler</code> <code>auth</code> , Zod parse
<code>service.ts</code>	SQL, transactions, business rules
<code>schemas.ts</code>	Zod request/response shapes

Portal routers in `api/src/routes/` **compose** module routes with prefixes — they do not contain business logic.

Example — customer portal (`routes/users-portal.ts`):

```

/v1/users/profile      → modules/users
/v1/users/carts/*      → modules/carts
/v1/users/purchases/*  → modules/purchases + documents
/v1/users/orders/*     → modules/orders
/v1/users/questionnaires/* → modules/questionnaires

```

4. API surface (portals)

All routes are under `/v1/` unless noted.

Prefix	Audience	Auth
<code>/health, /</code>	Ops	None
<code>/v1/auth</code>	All	Login/register public; <code>/session</code> JWT
<code>/v1/catalog</code>	Customer browse	Mostly public; reviews need customer JWT
<code>/v1/users</code>	Customer account	Customer JWT (<code>CUSTOMER</code> or <code>ADMIN</code>)
<code>/v1/help-center, /v1/legal</code>	Public CMS	None
<code>/v1/notifications</code>	In-app inbox	JWT
<code>/v1/public/consultations</code>	Guest video join	Token in URL
<code>/v1/doctor</code>	Doctor portal	Doctor JWT (<code>DOCTOR</code> or <code>ADMIN</code>)
<code>/v1/pharmacy</code>	Pharmacy staff	Pharmacy JWT (<code>PHARMACY_STAFF</code> or <code>ADMIN</code>)
<code>/v1/admin</code>	Admin CMS / ops	Admin JWT + permission checks
<code>/v1/webhooks</code>	Payment / courier	HMAC signature
<code>/v1/media, /v1/storage/user</code>	Uploads / downloads	Presign per role; guest presign for pre-cart photos
<code>/uploads/*</code>	Static (local dev)	Public local files

Catalog note: medicines grouped by subcategory (All Treatments) at `GET /v1/catalog/medicines/by-subcategory?categoryId=` .

Full route list: API.md (API.md). Patient subset: INTEGRATION.md (INTEGRATION.md).

5. Request lifecycle



Error handling

Central handler in `plugins/error-handler.ts` :

Source	HTTP	error code
ZodError	400	VALIDATION_ERROR
AppError subclasses	4xx	UNAUTHORIZED , FORBIDDEN , NOT_FOUND , ...
Unhandled	500	INTERNAL_ERROR

Typed errors live in `shared/errors.ts` (`UnauthorizedError` , `ForbiddenError` , ...).

6. Authentication layers

Authentication uses a **short-lived access JWT** plus an **opaque refresh token** stored (hashed) in PostgreSQL (`auth_refresh_tokens`).
 OTP codes are short-lived rows (`auth_otp_session` , `auth_email_otp_session`).

6.1 Login & token issuance

Email/password:

POST /v1/auth/login

- auth/service.login() – bcrypt verify, status check
- issueTokenPair() – access JWT + opaque refresh token (hash stored)

POST /v1/auth/register creates a CUSTOMER user and returns the same token pair.

Mobile OTP (primary customer app flow):

POST /v1/auth/otp/send → rate-limited OTP to mobile (mock or MSG91)
 POST /v1/auth/otp/verify → verify code; create customer if new; issue token pair
 POST /v1/auth/otp/complete-profile → name, gender, DOB, ethnicity (access JWT required); re-issue pair

OTP verify returns isNewUser and profileComplete. New users get placeholder email ({mobile}@otp.airohealth.local) until email is verified.

Refresh & logout:

POST /v1/auth/refresh → rotate refresh token; return new access + refresh pair
 POST /v1/auth/logout → revoke presented refresh token (body.refreshToken)

Refresh tokens are rotated on every successful refresh. Reuse of a revoked/rotated refresh token revokes **all** refresh tokens for that user (theft detection).

Email verification (profile email change):

POST /v1/auth/email-verification/send|resend|verify (JWT required)

Sends a 4-digit OTP via the notifications module (auth.email_verification_otp event → email channel).

Config: JWT_SECRET (min 16 chars), JWT_EXPIRES_IN (default 15m), REFRESH_TOKEN_EXPIRES_IN (default 30d), REFRESH_TOKEN_BYTES (default 32), OTP_PROVIDER , OTP_TTL_MINUTES , MSG91 keys when using SMS.

6.2 JWT payload

Defined in db/types.ts as JwtPayload :

Claim	When present	Use
sub	Always	User id
role	Always	CUSTOMER DOCTOR PHARMACY_STAFF ADMIN
doctorId	Doctors	Consultation/prescription scoping
pharmacyId	Pharmacy staff	Order/inventory scoping
pharmacyStaffRole	Pharmacy staff	Fine staff role (optional)
adminRoles	Admins	Permission matrix (see below)

Tokens are sent as:

Authorization: Bearer <accessToken>

6.3 Verification layer (`plugins/auth.ts`)

Method	Behavior
<code>app.authenticate</code>	<code>request.jwtVerify()</code> — rejects missing/invalid/expired token
<code>app.requireRoles([...])</code>	Authenticate + primary role must be in list

6.4 Authorization middleware (`shared/middleware.ts`)

Built on top of JWT verification:

Helper	Rule
<code>requireCustomer</code>	CUSTOMER or ADMIN
<code>requireCustomerProfileComplete</code>	Portal hook on <code>/v1/users</code> — blocks checkout mutations until profile fields complete (see <code>customerPathRequiresProfileComplete</code>)
<code>requireDoctor</code>	DOCTOR or ADMIN ; doctor accounts must have <code>doctorId</code> in JWT
<code>requirePharmacyStaff</code>	PHARMACY_STAFF or ADMIN ; must have <code>pharmacyId</code>
<code>requireAdmin</code>	ADMIN only
<code>requireAdminPermission('medicine', ...)</code>	Admin + scoped CMS permission
<code>requireSuperAdmin</code>	Admin + SUPERADMIN role row

Admin permission model (`modules/auth/admin-roles.ts`):

- Rows in `admin_user_roles` map users to roles: `MEDICINE` , `PHARMACY` , `DOCTOR` , `CUSTOMER_SUPPORT` , `CONTENT` , `REVIEW` , `SUPERADMIN` .
- Route groups use `requireAdminMedicine` , `requireAdminContent` , etc.
- **Legacy behavior:** admins with **no** role rows are treated as full access (all permissions). New admins should be assigned explicit roles.

Admin role	Typical scope
SUPERADMIN	All permissions + manage admin users
MEDICINE	Catalog CRUD, questionnaires, ingredients, subcategories
PHARMACY	Pharmacy onboarding, pharmacy order activity, courier sync
DOCTOR	Doctor onboarding, consultation list, reassignment
CUSTOMER_SUPPORT	Customer 360, orders, consultations, reassignment
CONTENT	Homepage, treatment pages, CMS blocks
REVIEW	Review moderation

Business-oriented role summary: BUSINESS_LOGIC.md §10 (BUSINESS_LOGIC.md).

6.5 Resource-level authorization (service layer)

Middleware only proves **who** the caller is. Services enforce **what** they can access:

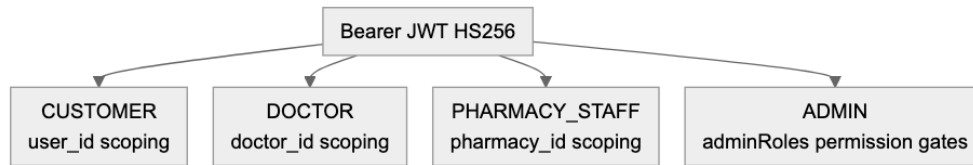
- Customer routes filter by `user_id = request.user.sub` .
- Pharmacy routes filter by `pharmacy_id = request.user.pharmacyId` .
- Doctor routes filter by `doctor_id = request.user.doctorId` .
- Admin routes bypass ownership but still respect permission middleware.

Example pattern: `getCustomerOrderPageDetail(userId, purchaseId)` joins through `user_purchases` so customers cannot read other users' orders even with a valid JWT.

6.6 Secondary tokens (not JWT)

Token	Purpose
Local upload HMAC	<code>signLocalUploadToken</code> — short-lived presign for dev uploads to API
Document download	Signed URL/token for PDF receipt/invoice download (<code>PURCHASE_DOCUMENT_URL_TTL_SECONDS</code>)
Pharmacy/doctor invite	<code>invite_token</code> on <code>doctors</code> / <code>pharmacies</code> rows (onboarding, not API session)
Mobile login OTP	4-digit code in <code>auth_otp_session</code> (not a JWT)
Email verification OTP	4-digit code in <code>auth_email_otp_session</code> (not a JWT)
Refresh token	Opaque random string; only SHA-256 hash in <code>auth_refresh_tokens</code>

6.7 Auth diagram



Not implemented today: OAuth/social login, API keys, mTLS. Access JWTs remain unrevocable until expiry; refresh tokens are revocable via `auth_refresh_tokens`.

7. Payments (Razorpay + dummy)

Mode	When	Flow
DUMMY	Local dev / tests	POST <code>/v1/users/purchases/:id/payment</code> with <code>outcome</code>
RAZORPAY	Production (when keys configured)	POST <code>.../razorpay/order</code> → client checkout → POST <code>.../razorpay/verify</code>

Razorpay webhook (POST `/v1/webhooks/razorpay`):

- Registered with a **raw body parser** so HMAC matches gateway bytes exactly.
- Verified via `x-razorpay-signature` + `RAZORPAY_WEBHOOK_SECRET`.
- Idempotent via `razorpay_webhook_events.event_id` (unique index).
- Side effects gated by `ENABLE_RAZORPAY_WEBHOOK=true`.

`user_purchases.provider_order_id` has a partial unique index — one Razorpay order maps to at most one purchase.

8. Notifications

Module: `api/src/modules/notifications/`.

```

Domain event (order placed, delivered, return, ...)
  → hooks.ts
  → notification-service.dispatchNotification()
  → event-routing (channels per event)
  → providers (mock / meta / ses / msg91 / fcm)
  → notification_log (audit)
  
```

Event	Typical channels
order.placed	WhatsApp, email, SMS
delivery.dispatched , delivery.out_for_delivery	WhatsApp, SMS
order.delivered , order.accepted , order.cancelled	WhatsApp, email, SMS
return.submitted , return.refunded	WhatsApp, email, SMS
prescription.approved	WhatsApp, email
auth.email_verification_otp	Email

Providers are selected per channel via env (`NOTIFICATION_WHATSAPP_PROVIDER` , `NOTIFICATION_EMAIL_PROVIDER` , etc.). Failures are logged but do not block the calling transaction (`notifyAsync`).

9. Data layer

9.1 PostgreSQL

- **Pool:** `db/pool.ts` — max 20 connections, 5s connect timeout.
- **BIGINT ids** parsed as JavaScript numbers (safe for current id range).
- **Schema:** `database/initial.sql` (full schema; reset via `./scripts/seed-dummy-data.sh` for local dev).
- **No ORM** — hand-written SQL in services; transactions via `pool.connect()` + `BEGIN` / `COMMIT` where needed.

9.2 Domain data flows (simplified)

```

Browse catalog          → medicines, conditions, page_content_blocks
Cart & questionnaire    → user_carts, user_questionnaire_sessions
Checkout               → checkout session, user_purchases (OTC + RX
split when mixed)
Payment success        → fulfillCheckout (idempotent)
    └─ OTC: auto-prescription → fulfillment planner → pharmacy_order
    └─ RX: consultation + random doctor assign → reserve stock →
pharmacy_order on Rx issue
Doctor                 → user_prescription →
forwardPharmacyOrdersForPrescription
Pharmacy               → shipment, purchase_documents (invoice/receipt)
Customer orders API    → purchase id + displayStatus/timeline builders
Returns                → order_return_request → status sync

```

Key cross-cutting builders:

- `modules/orders/order-track-timeline.ts` — customer track UI (`displayStatus` , `timeline` steps, return rails).
- `modules/inventory/fulfillment-service.ts` + `fulfillment-planner.ts` — post-payment pharmacy assignment (see §9.3).
- `modules/purchases/checkout-service.ts` — mixed-cart split, doctor pick, OTC auto-Rx.

- `modules/content/homepage-curation.ts` — resolves CMS `reviewIds` / `medicineIds` into API payloads.

9.3 Fulfillment & assignment architecture

Technical counterpart to BUSINESS_LOGIC.md §7a–7b (BUSINESS_LOGIC.md).

Doctor assignment (RX payment)

Module: `modules/purchases/checkout-service.ts`

```
fulfillCheckout (RX path)
  → resolveCartPrimaryCategoryId(cartId)  // first Rx item category,
  else first item
  → pickRandomDoctorForCategory(categoryId)
    // ACTIVE doctors JOIN doctor_categories WHERE category_id = ?
    // ORDER BY RANDOM() LIMIT 1
  → INSERT user_consultations (doctor_id, ...)
  → emitConsultationAssigned + scheduleClinicalAiForConsultation
```

Manual override: `PATCH /v1/admin/consultations/:id/doctor` — any **ACTIVE** doctor (`activity-service.assignAdminConsultationDoctor`); category link not required.

Pharmacy assignment (payment)

Modules: `modules/inventory/fulfillment-planner.ts` , `fulfillment-service.ts` , `pharmacy-proximity.ts`

```
assignPurchaseFulfillment(purchaseId)
  → load lines + delivery pincode/city/state
  → when FULFILLMENT_RANKING=shiprocket:
    scorePharmaciesForDelivery() per candidate pharmacy pincode
    // Shiprocket serviceability API; Redis cache 24h; 3s timeout
  per call
  → planFulfillmentAssignments(lines, pharmacyInventory, delivery,
  ranking)
    // Prefer SINGLE pharmacy fulfilling all lines
    // Rank (shiprocket): serviceable > lowest ETD > lowest rate >
  tier > stock > pharmacy id
    // Rank (tier fallback): pincode > city+state > state > other;
  tie-break stock qty
    // Else SPLIT across pharmacies (greedy cover, same ranking)
    // Else null → UNFULFILLABLE
  → reserveInventoryAtPharmacy per assignment
  → pharmacy_order created when prescription ISSUED (RX) or
  immediately (OTC auto-Rx)
```

Env: `FULFILLMENT_RANKING=tier|shiprocket` (defaults to `shiprocket` when `COURIER_PROVIDER=shiprocket`).

Patient-facing behavior: BUSINESS_LOGIC.md §7b (BUSINESS_LOGIC.md).

10. Storage & media

Controlled by `STORAGE_DRIVER` :

Mode	Upload	Catalog download	User download
local (dev)	Presigned PUT → API → disk	<code>UPLOAD_PUBLIC_BASE_URL/...</code>	HMAC local download URL
s3 (prod)	Presigned PUT → S3	CloudFront only	Fresh Pre-signed GET only

S3 env (production)

```
STORAGE_DRIVER=s3
AWS_REGION=ap-south-1
CATALOG_BUCKET=airoemed-catalog-prod
USER_BUCKET=airoemed-user-prod
CATALOG_CDN_URL=https://cdn.airoemed.com
```

Bucket	Folders / keys	Download
CATALOG_BUCKET	categories, subcategories, ingredients, medicines, doctors/*, pharmacies/logos, reviews, admin uploads	Always CloudFront (<code>CATALOG_CDN_URL/{objectKey}</code>) — never Pre-signed GET
USER_BUCKET	carts, questionnaires, questionnaire-photos, returns, consultations, profile, purchase PDFs (documents/purchases/...)	Always Pre-signed GET — never CloudFront

Bucket choice is by **folder** (`resolveMediaBucketKind` in `folder-policy.ts`), not by role.

Services

- `CatalogStorageService` — `buildPublicUrl()`, `deleteObject()`
- `UserStorageService` — `generateDownloadUrl()`, `deleteObject()`

Persist in DB

Store `bucket` + `objectKey` (+ optional `contentType` / `size` / `originalFileName`). Never store CloudFront URLs, S3 URLs, or Pre-signed URLs.

Presign upload flow (unchanged PUT path)

1. Client calls `POST /v1/{portal}/media/presign` with folder, fileName, mimeType, size.
2. `folder-policy.ts` enforces allowed folders per role and maps folder → catalog/user bucket.
3. API returns `uploadUrl` + `objectKey` / `fileKey`. Catalog also returns `publicUrl` (CloudFront).

4. In dev (`MEDIA_DIRECT_S3_PRESIGN=false`), browser PUTs to API `/v1/media/local-upload` with HMAC token instead of S3.

User private download

1. After upload, register metadata: `POST /v1/storage/user/attachments`
2. Authenticated download: `GET /v1/storage/user/download-url/{attachmentId}` → fresh `{ downloadUrl }`

Purchase documents (receipts/invoices): generated PDF → `USER_BUCKET` (or local `uploads/documents/`) → `storage_key` in `purchase_documents` → Pre-signed GET via `UserStorageService` (or local HMAC token).

11. Notable cross-cutting concerns

Concern	Implementation
Order status mapping	<code>shared/order-status.ts</code> — DB enums → public API strings
Return window	<code>shared/order-return-window.ts</code> — 72h after delivery
Customer order id	<code>shared/public-ids.ts</code> — public display <code>OD#####</code> (<code>ORDER_PUBLIC_PREFIX</code> + public code body)
Slugs	<code>shared/slugify.ts</code> + per-entity slug utils
Stock / availability	<code>modules/catalog/medicine-availability.ts</code> — inventory-derived <code>IN_STOCK</code> / <code>LOW_STOCK</code> / <code>OUT_OF_STOCK</code>
Email (invites)	<code>shared/sendPortalCredentialsEmail.ts</code>
Order notifications	<code>modules/notifications/hooks.ts</code> — fired from purchases, orders, shipments, returns, prescriptions
OTP rate limiting	<code>auth_otp_send_log</code> / <code>auth_email_otp_send_log</code> + env window limits

12. Deployment

```

docker compose
├─ db          PostgreSQL 16 (local dev; RDS in production)
├─ redis       Redis 7 (catalog response cache)
├─ api         telemed-api (port 3000)
└─ clinical-ai optional doctor-assist service

```

- Production API is fronted by `nginx` (`nginx/airoemed.conf`) terminating TLS and proxying to the API container.
- Graceful shutdown: `SIGINT` / `SIGTERM` → `app.close()` + `pool.end()` .

Detailed CI/CD and host secrets: `internal ci-runner-setup.md` (`ci-runner-setup.md`) (ops team).

13. Caching layer

13.1 Why caching

Pain point	Cache benefit
Homepage + PDP browse traffic	Reduce repeated joins on <code>medicines</code> , <code>conditions</code> , CMS blocks
Catalog browse	Lower Postgres load for public GETs
Search	Cached popular query results

Do not cache: carts, purchases, orders, consultations, live inventory used for checkout, auth, user-specific prefill.

13.2 Architecture (implemented)

```

service / catalog route
├── cache.getOrSet(key) —hit→ return (+ live availability overlay)
└── miss → PostgreSQL → Redis SET EX TTL → return

```

Stack: Redis 7 (`REDIS_URL`), client `ioredis`, module `api/src/cache/`.

Keys: version-prefixed `v{N}:catalog:...` . `catalog:version` bumped on admin catalog writes and via `POST /v1/admin/cache/clear`.

Default TTL: `CACHE_DEFAULT_TTL_SECONDS` (minimum **3600** / 1 hour). Freshness is primarily via admin clear / version bump.

Fail-open: if Redis is down or unset, loaders hit Postgres directly.

13.3 Cache candidates

Key family	TTL	Notes
Homepage, categories, concerns, medicines, doctors, reviews, CMS, search, questionnaire definitions	≥1h	Live availability attached after cache
Availability	not cached (or ≤30s if added later)	Checkout must stay fresh

13.4 Invalidation

- Admin API:** `POST /v1/admin/cache/clear { "scope": "all"|"catalog"|"homepage"|"medicines"|"doctors"|"search" }` — bumps catalog version.
- Write-through:** successful admin mutations under catalog-related prefixes bump version automatically.
- Customer review POSTs also bump version.

13.5 HTTP caching

Public catalog GETs set `Cache-Control: public, max-age=60, stale-while-revalidate=300` . Homepage supports weak `ETag / If-None-Match` .

13.6 JWT & caching

Refresh-token revocation remains the logout control. JWT denylist in Redis is not part of this rollout.

13.7 Ops

Admins clear catalog cache via `POST /v1/admin/cache/clear` with body `{ "scope": "all"|"catalog"|"homepage"|"medicines"|"doctors"|"search" }` .

Env: `REDIS_URL` , `CACHE_ENABLED` , `CACHE_DEFAULT_TTL_SECONDS=3600` .

14. Testing & observability

Area	Today
Unit tests	Selected modules (order timeline, notifications, public ids)
Integration tests	Postman collections in <code>api/postman/</code>
Logging	Structured JSON; <code>x-request-id</code> correlation; secrets redacted
Metrics / APM	Optional OpenTelemetry when configured
Health	<code>GET /health</code> — process + database (+ Redis when configured)

15. Current capabilities & known limitations

Area	Status
Razorpay payments	Implemented — checkout, verify, webhook (dummy gateway for dev)
Mobile / email OTP	Implemented — mock + MSG91
Multi-channel notifications	Implemented — WhatsApp, email, SMS, in-app inbox
Redis catalog cache	Implemented — admin clear API
Order timeline API	Implemented — <code>displayStatus</code> + <code>timeline</code> for patient apps
Clinical AI assist	Implemented — async, doctor-facing
Real-time patient order updates	Polling REST + push notifications (no WebSocket to patient app)
Discount / promo codes	Not implemented
OAuth / social login	Not implemented
Unfulfillable orders	Marked <code>UNFULFILLABLE</code> — manual admin refund required
JWT revocation	Refresh tokens revocable; access JWT valid until expiry
Background job queue	PDF gen, Clinical AI, notifications largely inline / fire-and-forget today

Business-facing summary of gaps: BUSINESS_LOGIC.md §13 (BUSINESS_LOGIC.md).

16. Security checklist

- Passwords hashed with bcrypt; never returned in API responses.
- Helmet enabled; CORS restricted to configured origins.
- JWT secret required at startup; upload tokens signed with same secret (consider separate `UPLOAD_SIGNING_SECRET` in future).
- Role + resource scoping in services for multi-tenant pharmacy/doctor data.
- Presign uploads restricted by role folder policy and MIME allowlist.
- SQL uses parameterized queries (`$1` , `$2`) throughout services.
- Razorpay webhook trust is HMAC-only; raw body preserved for signature verification.
- OTP send endpoints are rate-limited per mobile/user window.
- Customer checkout paths require completed profile (**name**, **gender**, **DOB**; ethnicity optional).

Last updated to reflect the codebase under `telemed-backend/api` and `database/` . See also BUSINESS_LOGIC.md (BUSINESS_LOGIC.md) and INTEGRATION.md (INTEGRATION.md).